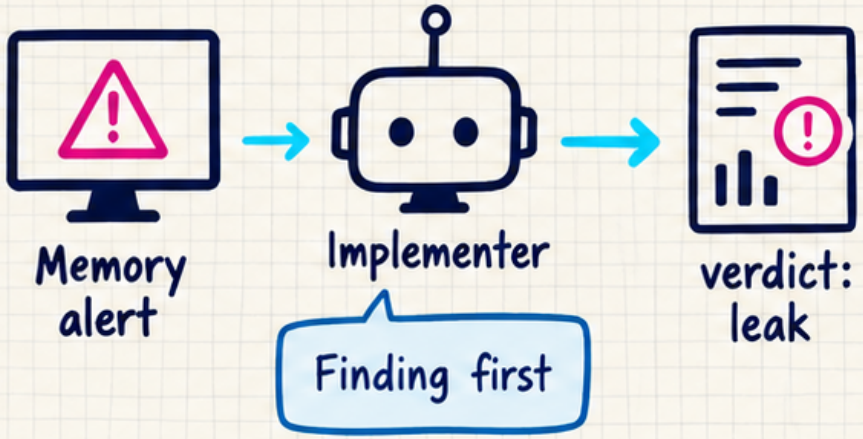
A LEAK BECOMES A HELD ACTION

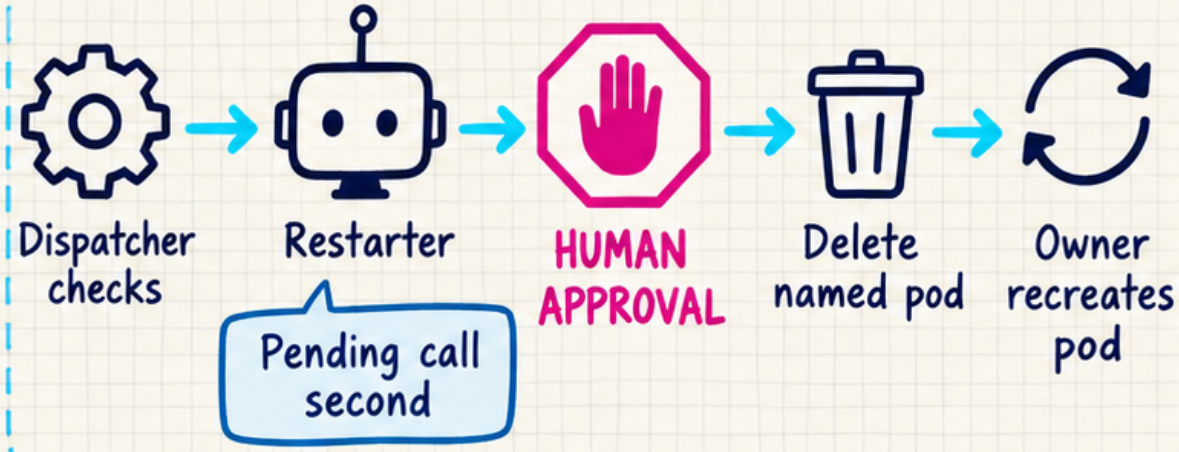
01/08. Diagnosis is automatic. A restart waits for a human.

Session 1: evidence



New session + evidence reference

Session 2: action



The dispatcher never approves.

Restart relieves a symptom; it does not fix the leak.

ONE VERDICT LINE CHOOSES THE PATH

Completed implementer answer

verdict: leak

Parse the verdict line, not the prose.

Read the raw answer before truncation or scrubbing.

limit-too-low

Existing resource-fix PR path

leak

Memory row may request a gated restart.

contention

Finding only.

inconclusive

Finding only.

Missing or unknown

↓

null

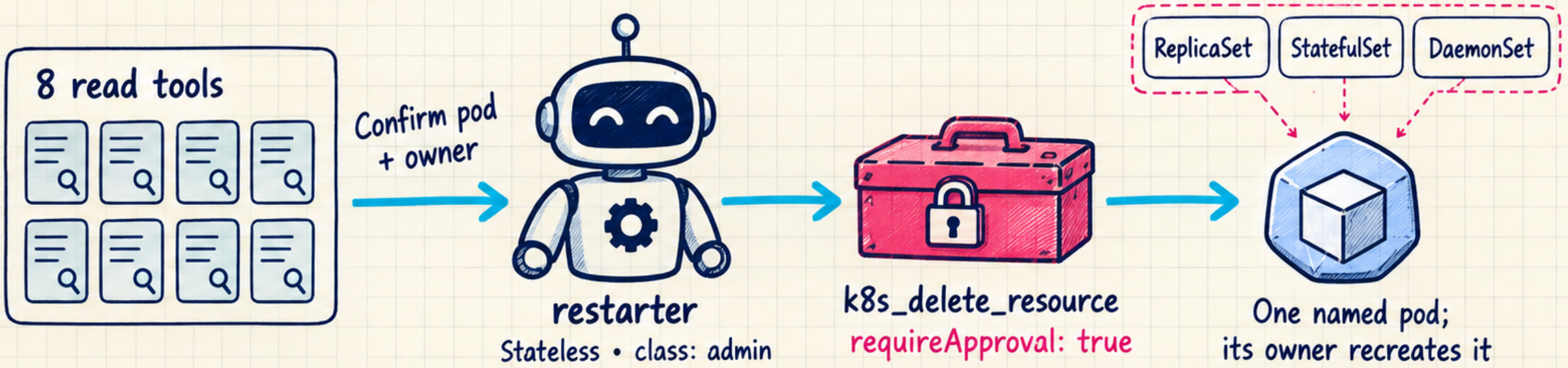
↓

no remediation.

The verdict is the diagnosis, not the action.

A fix already in Git can still be limit-too-low.

THE RESTARTER HAS ONE MUTATING TOOL



Refuse an ownerless pod.

Refuse another target or resource kind.

Reject means stop.

Admin class permits the destructive binding; approval is still required.

No scale, patch or edit tools.

CHECK THE TARGET BEFORE OFFERING APPROVAL

01 BEFORE THE SESSION

- Valid namespace + pod labels
- Namespace not on deny list

PASS

02 AFTER THE PAUSE

- Exactly one pending call
- Tool: k8s_delete_resource
- Type: pod
- Name + namespace match the alert

FAIL

✗ skip-target • no session

PASS

✓ Show approve + reject

FAIL

✗ MISMATCH → reject only

One --approve approves every pending call; inspect the exact arguments.

→ **KEY TAKEAWAY:** The prompt is a fence. Dispatcher checks add independent checks.

⚠ **CAVEAT:** MISMATCH suppresses the offered command; it is not a new authorization layer.

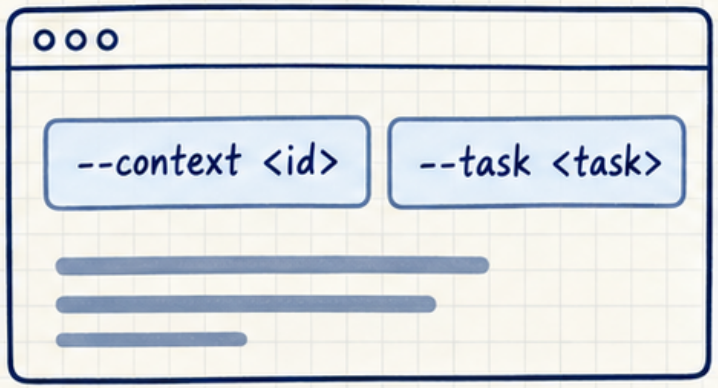
APPROVE THE TASK THAT YOUR LAPTOP NEVER SAW

Read the Slack evidence and pending arguments first

01 Copy the supplied command

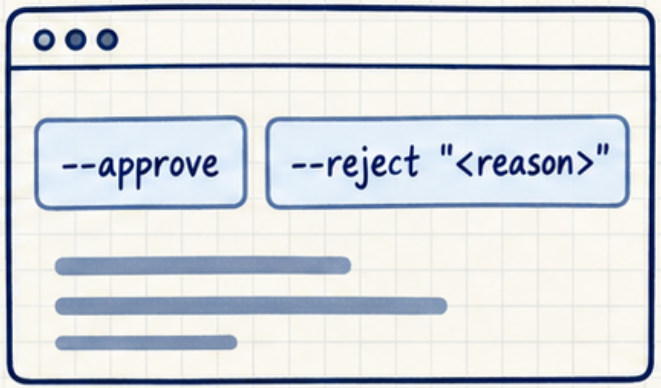
Agent: kagent/restarter
Identity: dispatcher/dispatcher

02 Fetch the paused task



A2A GetTask loads the pending request

03 Approve or reject



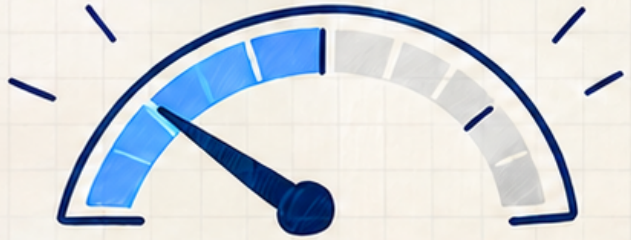
Use the complete command from Slack; full templates are in the notes.

The kagent UI cannot see this dispatcher-owned session.

The dispatcher opens the session. A human decides the pending call.

Two Budgets Prevent Restart Loops

GLOBAL CAP



2 restarter sessions per rolling 24 hours

Across all remediation rows.

RECURRENCE CAP

Once per namespace/workload in 24 hours

Replacement pod names do not reset the key.

Target passes checks. → Check + charge budget. → Admitted → Open restarter session.

Check + charge budget. → skip-restarts → cap reached.

Check + charge budget. → skip-recurring → upstream fix needed.

Refusal, rejection, error or timeout still consumes the slot.

Budgets are in memory; dispatcher restart resets them.

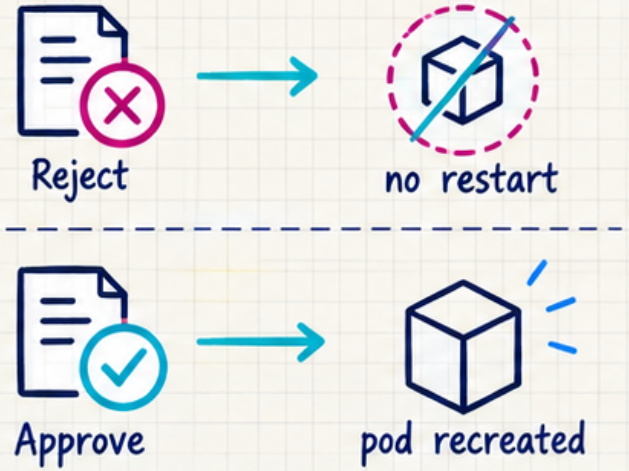
Count requested sessions, not successful restarts.

WHAT THE ACCEPTANCE ACTUALLY PROVED

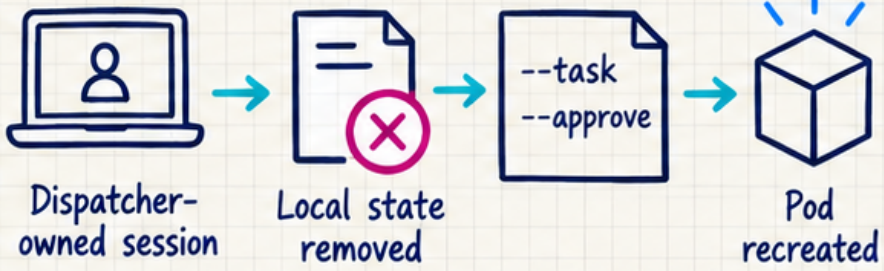
07/08

Recorded • 14 OCT 2026

LIVE: DIRECT RESTARTER



LIVE: REMOTE APPROVAL



LIVE: HEALTHY ALERT REPLAY

TESTED: leak verdict → second session → pending approval

No real leaking pod was available for end-to-end replay.

→ Live components + tested routing; no invented full leak demo.

A RESTART IS RELIEF, NOT A REPAIR

ATTRIBUTION
Audit records approval as dispatcher identity, not the human approver.

TIMEOUT
A charged session can leave an unreachable pending call.

AGENT CONTRACT
Target and one-delete rules remain prompt constrraints.

Secret scrubbing is best-effort.

Contention stays a finding; no mechanical restart route.

Keep the evidence, the approval and the remaining limits visible.